

Decoupled consensus

1 Common substrate

A fixed validator set V is given. Slot s starts at $t_s = 4\Delta s$ and has an assigned proposer and a fixed committee $\mathcal{K}_s \subseteq V$. After t_{GST} , Δ is a strict delivery bound, and an honest node relays every object it processes. A scheduled action uses the store immediately before its public time.

Every block B has a root $B.\text{root}$, slot $B.\text{slot}$, and parent $B.\text{parent}$. Processed blocks form a parent-closed tree rooted at B_{gen} . Write $B \preceq C$ when $B = C$ or B is an ancestor of C , and write $B \prec C$ for strict ancestry. Two blocks are compatible when one is an ancestor of the other, and they conflict otherwise. The depth of a block is the number of parent edges from genesis. “Deepest” means maximum depth; ties use a fixed root order.

$\Sigma.\text{timestamp}(x)$ is the time at which object x is processed into the store.

2 Goldfish available chain

This section defines block-only Goldfish: the store, view merge, one-slot fork choice, and available confirmation. Later sections add fields to this store and add lines to these handlers; they never rename either.

2.1 Schedule and wire objects

Slot s has these public actions:

t_s	proposal,
$t_s + \Delta$	Goldfish vote,
$t_s + 2\Delta$	support cutoff,
$t_s + 3\Delta$	view freeze,
$t_s + 6\Delta$	slot- s confirmation evaluation.

Because $t_s + 6\Delta = t_{s+1} + 2\Delta$, the last action is also the support action of slot $s + 1$.

A Goldfish vote is a tuple (v, s, B) from validator $v \in \mathcal{K}_s$ with target B , where $B.\text{slot} \leq s$. A block B carries $B.\text{parent}$, $B.\text{slot}$, and a set $B.\text{gf_votes}$ of slot- $(B.\text{slot} - 1)$ Goldfish votes. There is no proposal envelope: the block is the only wire object a proposer emits, and $B.\text{gf_votes}$ is the only relay channel in the protocol.

2.2 Store

Definition 1 (Store). The store is

$$\Sigma = (t, s, \mathcal{T}, \text{timestamp}[\cdot], \text{gf_votes}[\cdot], \text{live_confirmed}, \text{latest_confirmed}).$$

$\Sigma.\mathcal{T}$ is the tree of processed blocks and $\Sigma.\text{gf_votes}[k]$ the set of processed slot- k votes. Both are timestamped. $\Sigma.\text{gf_votes}[k]$ keeps at most two distinct votes per validator, which is all any rule reads.

$\Sigma.\text{live_confirmed}$ is the block the last evaluated slot confirmed. $\Sigma.\text{latest_confirmed}$ is the monotone record the node exposes; no rule in this protocol reads it.

Initially $\Sigma.\mathcal{T} = \{B_{\text{gen}}\}$, $\Sigma.\text{timestamp}(B_{\text{gen}}) = -\infty$, and $\Sigma.\text{live_confirmed} = \Sigma.\text{latest_confirmed} = B_{\text{gen}}$; other fields are empty.

The store keeps messages and their arrival times, and nothing else. Every rule below is a timestamp comparison on this one pool. A validator has equivocated as of t when $\Sigma.\text{gf_votes}[k]$ holds two of its votes both timestamped before t , so its *equivocation-detection time* is the later of the two stamps, and one cutoff decides which votes are timely and which equivocations are.

$\text{PROCESS_BLOCK}(\Sigma, B)$ and $\text{PROCESS_GOLDFISH_VOTE}(\Sigma, \text{vote})$ run once per object, after every dependency of that object is already in the store: a block's parent, and a vote's target block. A carried vote may name a block the receiver does not hold; acquiring it is outside the store, so that vote reaches the store when it is finally processed, not when its block did.

2.3 View merge

To run the Goldfish fork-choice in slot s , a voter at $t_s + \Delta$ uses the slot- $(s-1)$ votes it saw before the "freeze time" (based on the timestamp) at $t_{s-1} + 3\Delta$, together with the votes from the current block proposal, if there is one. Equivocators (validators with two votes in the pool) are counted as voting for every block, which gives an equivalent output to removing them.

The proposer does not apply the freeze, and instead simply uses all votes in the pool when running the fork-choice at t_s .

2.4 Goldfish fork-choice

Definition 2 (Goldfish score and walk). Fix a vote slot s and a set votes of slot- s votes. Validator $v \in \mathcal{K}_s$ *equivocates* when votes holds two of its distinct votes, and *participates* when it holds at least one; write $N_s(\text{votes})$ for the participants. $\text{GOLDFISH_SCORE}(\text{votes}, s, B)$ counts every equivocator plus every non-equivocating participant whose target descends from B . An equivocator counts for every block and stays among the participants, so it can neither create nor block a descent. A non-equivocating validator counts once, in one subtree.

Throughout this document, we use this as a building block:

$$\text{GHOST}(\text{anchor}, \text{tree}, \text{score}, \text{eligible}),$$

where score is a function on blocks and eligible a predicate on blocks. It descends from anchor through eligible children in tree , taking the highest score at each step, and stops where no child is eligible.

Goldfish instantiates the walk with $\text{GOLDFISH_SCORE}(\text{votes}, s, \cdot)$ and the gate

$$\begin{aligned} \text{GOLDFISH_ELIGIBLE}(\Sigma, \text{votes}, s, B) \iff & 2 \text{ GOLDFISH_SCORE}(\text{votes}, s, B) > \text{voters_count} \\ & \text{or } B.\text{slot} = \Sigma.s, \end{aligned}$$

with $\text{voters_count} = |\{v \in \mathcal{K}_s : \text{votes} \text{ holds a vote by } v\}|$. The majority gate enforces timeliness and other fork-choice conditions, like FOCIL inclusion. It only does not apply to proposals from the current slot, which cannot yet have votes.

At slot 0 the vote set is empty, no child is eligible, and the head is genesis.

Figure 1: Goldfish score and walk

```

1: function GOLDFISH_SCORE( $\text{votes}, s, B$ )
2:    $\text{equivocators} \leftarrow \{v \in \mathcal{K}_s : \text{votes} \text{ holds two distinct votes by } v\}$ 
3:    $\text{supporters} \leftarrow \{v \in \mathcal{K}_s \setminus \text{equivocators} : (v, s, B') \in \text{votes} \text{ with } B \preceq B'\}$ 
4:   return  $|\text{equivocators}| + |\text{supporters}|$  ▷ equivocators count for every block
5: function GHOST( $\text{anchor}, \text{tree}, \text{score}, \text{eligible}$ )
```

```

6:   $H \leftarrow \text{anchor}$ 
7:  loop
8:     $\text{children} \leftarrow \{B : B \text{ is a child of } H \text{ in } \text{tree}, \text{eligible}(B)\}$ 
9:    if  $\text{children} = \emptyset$  then
10:      return  $H$ 
11:     $H \leftarrow \arg \max_{B \in \text{children}} \text{score}(B)$ , ties by root order
12: function  $\text{GOLDFISH\_ELIGIBLE}(\Sigma, \text{votes}, s, B)$ 
13:    $\text{voters\_count} \leftarrow |\{v \in \mathcal{K}_s : \text{votes holds a vote by } v\}|$ 
14:   return  $2 \text{GOLDFISH\_SCORE}(\text{votes}, s, B) > \text{voters\_count}$  or  $B.\text{slot} = \Sigma.s$ 
15: function  $\text{GOLDFISH\_FORK\_CHOICE}(\Sigma, \text{anchor}, \text{tree}, \text{votes}, s)$ 
16:   return  $\text{GHOST}(\text{anchor}, \text{tree}, \text{GOLDFISH\_SCORE}(\text{votes}, s, \cdot), \text{GOLDFISH\_ELIGIBLE}(\Sigma, \text{votes}, s, \cdot))$ 
17: function  $\text{GET\_HEAD}(\Sigma, \text{votes}, s)$ 
18:   return  $\text{GOLDFISH\_FORK\_CHOICE}(\Sigma, B_{\text{gen}}, \Sigma.\mathcal{T}, \text{votes}, s)$ 

```

2.5 Duties and handlers

Figure 2 defines the store handlers and the two duties. The handlers take the store and change it in place. Only the two duties concern the validator running the node, written ℓ ; a node whose ℓ holds no duty for the slot simply does not run them.

Figure 2: Goldfish duties and store handlers

```

1: function  $\text{ON\_TICK}(\Sigma, t)$ 
2:    $s \leftarrow \lfloor t/(4\Delta) \rfloor$ ;  $(\Sigma.t, \Sigma.s) \leftarrow (t, s)$ 
3:   if  $s > 0$  and  $t = t_s$  and  $\ell$  is the slot- $s$  proposer then
4:      $\text{PROPOSE\_BLOCK}(\Sigma)$ 
5:   if  $s > 0$  and  $t = t_s + \Delta$  then
6:      $\text{GOLDFISH\_VOTE}(\Sigma)$ 
7:   if  $s > 0$  and  $t = t_s + 2\Delta$  then
8:      $\text{UPDATE\_CONFIRMATION}(\Sigma, s - 1)$ 
9: function  $\text{PROCESS\_BLOCK}(\Sigma, B)$ 
10:  if  $B.\text{slot} > \Sigma.s$  then
11:    return
12:  add  $B$  to  $\Sigma.\mathcal{T}$ ;  $\Sigma.\text{timestamp}(B) \leftarrow \Sigma.t$ 
13:  for all  $\text{vote} \in B.\text{gf.votes}$  do
14:     $\text{PROCESS\_GOLDFISH\_VOTE}(\Sigma, \text{vote})$ 
15: function  $\text{PROCESS\_GOLDFISH\_VOTE}(\Sigma, \text{vote})$ 
16:  if  $\text{vote.slot} > \Sigma.s$  or  $\text{vote} \in \Sigma.\text{gf.votes}[\text{vote.slot}]$  then
17:    return
18:  if  $\Sigma.\text{gf.votes}[\text{vote.slot}]$  holds two distinct votes by  $\text{vote.validator}$  then
19:    return ▷ two witness the equivocation; nothing reads a third
20:  add  $\text{vote}$  to  $\Sigma.\text{gf.votes}[\text{vote.slot}]$ ;  $\Sigma.\text{timestamp}(\text{vote}) \leftarrow \Sigma.t$ 
21: function  $\text{PROPOSE\_BLOCK}(\Sigma)$ 
22:   $s \leftarrow \Sigma.s$  ▷ runs at  $t_s$ 
23:   $\text{votes} \leftarrow \Sigma.\text{gf.votes}[s - 1]$  ▷ everything held: the tick precedes anything timestamped  $t_s$ 
24:   $H \leftarrow \text{GET\_HEAD}(\Sigma, \text{votes}, s - 1)$ 
25:   $B \leftarrow$  a block with  $B.\text{parent} = H$ ,  $B.\text{slot} = s$ ,  $B.\text{gf.votes} = \text{votes}$  ▷ everything, not only the pre-freeze part
26:  broadcast  $B$ ;  $\text{PROCESS\_BLOCK}(\Sigma, B)$ 
27: function  $\text{GOLDFISH\_VOTE}(\Sigma)$ 
28:   $s \leftarrow \Sigma.s$  ▷ runs at  $t_s + \Delta$ 
29:   $\text{votes} \leftarrow \{\text{vote} \in \Sigma.\text{gf.votes}[s - 1] : \Sigma.\text{timestamp}(\text{vote}) < t_{s-1} + 3\Delta\}$  ▷ held before the freeze
30:  for all  $B \in \Sigma.\mathcal{T}$  with  $B.\text{slot} = s$  do
31:     $\text{votes} \leftarrow \text{votes} \cup B.\text{gf.votes}$  ▷ view merge
32:   $H \leftarrow \text{GET\_HEAD}(\Sigma, \text{votes}, s - 1)$ 
33:  if  $\ell \in \mathcal{K}_s$  then
34:     $\text{vote} \leftarrow (\ell, s, H)$ ; broadcast  $\text{vote}$ ;  $\text{PROCESS\_GOLDFISH\_VOTE}(\Sigma, \text{vote})$ 

```

2.6 Available confirmation

Confirmation is the same walk over a stricter vote set and a larger denominator. Let *early* and *late* be the slot-*s* votes timestamped before $t_s + 2\Delta$ and before $t_s + 6\Delta$. The walk scores

$$votes = \{vote \in early : vote.validator \text{ does not equivocate in } late\}$$

against the participant count of *late*. A validator counts when it voted in time and no second vote of its has appeared since; the denominator counts everyone who voted at all.

Because $early \subseteq late$, a validator equivocating in *early* equivocates in *late* too, so *votes* holds at most one vote per validator and the equivocator clause of the score never fires here. At most one child can pass the gate, so the descent has no choice to make.

Slot *s* is evaluated once, at $t_s + 6\Delta$, from genesis over the live tree. The result is at worst genesis, never empty.

Figure 3: Available confirmation

```

1: function UPDATE_CONFIRMATION( $\Sigma, s$ ) ▷ at  $t_s + 6\Delta$ 
2:    $early \leftarrow \{vote \in \Sigma.gf\_votes[s] : \Sigma.timestamp(vote) < t_s + 2\Delta\}$ 
3:    $late \leftarrow \{vote \in \Sigma.gf\_votes[s] : \Sigma.timestamp(vote) < t_s + 6\Delta\}$ 
4:    $votes \leftarrow \{vote \in early : late \text{ holds no second distinct vote by } vote.validator\}$ 
5:    $voters\_count \leftarrow |\{v \in \mathcal{K}_s : late \text{ holds a vote by } v\}|$ 
6:   define  $eligible(B)$  as  $2 \text{ GOLDFISH\_SCORE}(votes, s, B) > voters\_count$ 
7:    $H \leftarrow \text{GHOST}(B_{gen}, \Sigma, \mathcal{T}, \text{GOLDFISH\_SCORE}(votes, s, \cdot), eligible)$ 
8:    $\Sigma.live\_confirmed \leftarrow H$ 
9:   if  $\Sigma.latest\_confirmed \preceq H$  then
10:     $\Sigma.latest\_confirmed \leftarrow H$ 

```

3 Relative-majority stabilization gadget

SG means *stabilization gadget*. This intermediate protocol adds one message layer to Goldfish: a relative-majority fork choice over the latest SG votes selects the root from which the Goldfish walk starts.

3.1 Rounds and SG votes

Each validator $v \in V$ has a fixed positive integer weight $w(v)$. For $S \subseteq V$, write $w(S) = \sum_{v \in S} w(v)$ and $W = w(V)$.

For a fixed integer $R \geq 1$, round r consists of slots $rR, \dots, rR + R - 1$; slot rR is its *opening slot*, and $\text{round}(s) = \lfloor s/R \rfloor$. Each round has one SG vote time a_r , a public parameter in this intermediate protocol.

An SG vote is a tuple (v, r, H) from validator $v \in V$ with head H , a block or $\perp$. At a_r , an honest validator votes its current *live_confirmed*, which is a block; the empty head appears only in adversarial votes here, and in the timeout votes of the graded protocol. A round- r vote is read from round $r + 1$ on.

3.2 Store

This layer adds one field: $\text{sg_votes}[r]$, the set of processed round- r SG votes. Like gf_votes , it is timestamped and keeps at most two distinct votes per validator, which is all any rule reads; two distinct heads from one validator in one round are an equivocation. SG votes travel only on the wire. Blocks do not carry them, and they never enter a Goldfish vote set.

3.3 Latest votes and majority root

Definition 3 (Latest vote, support, and majority root). Fix an expiry window $\eta_{\text{SG}} \geq 1$ in rounds. For validator v and round r , $\text{LATEST}(\Sigma, v, r)$ is the greatest round $k \in [\max\{0, r - \eta_{\text{SG}}\}, r)$ in which

$\text{sg_votes}[k]$ holds a vote by v , or $\perp$ when there is none. A validator with a latest round is *represented*: it counts in the denominator whatever its votes say. It *supports* a block only when its latest round holds exactly one distinct vote by it and that vote's head is a block. An empty or equivocating latest round therefore supplies no support and, because only the latest round is read, also silences every older head; a later clean round restores support.

$\text{SG_SUPPORT}(\Sigma, r, B)$ is the represented weight supporting B . The entire represented weight is $W_r = w(\{v \in V : \text{LATEST}(\Sigma, v, r) \neq \perp\})$. Ancestry is read in the live tree $\mathcal{T}$, so a head outside a restricted child tree still supports the child through which it descends. $\text{MAJORITY_FORK_CHOICE}$ runs the walk with this score, gated on a strict majority of W_r . Because an equivocator supplies no support but stays in the denominator, and conflicting children have disjoint supporter sets, two conflicting children cannot both pass: the descent is uniquely determined, and equivocating weight can only raise the bar.

The composed head runs Goldfish from the majority root instead of genesis (Figure 4): the SG walk selects the anchor, and the Goldfish walk selects a descendant of it. The two walks are the same function with different scores and gates.

Figure 4: SG fork choice

```

1: function LATEST( $\Sigma, v, r$ )
2:    $\text{eligible} \leftarrow \{k : \max\{0, r - \eta_{\text{SG}}\} \leq k < r, \text{sg\_votes}[k] \text{ holds a vote by } v\}$ 
3:   if  $\text{eligible} = \emptyset$  then
4:     return  $\perp$ 
5:   return  $\max \text{eligible}$ 
6: function SG_SUPPORT( $\Sigma, r, B$ )
7:    $\text{supporters} \leftarrow \emptyset$ 
8:   for all  $v \in V$  with  $\text{LATEST}(\Sigma, v, r) \neq \perp$  do
9:      $k \leftarrow \text{LATEST}(\Sigma, v, r)$ 
10:    if  $\text{sg\_votes}[k]$  holds exactly one distinct vote  $(v, k, H)$  by  $v$ , with  $H \neq \perp$  and  $B \preceq H$  then
11:       $\text{supporters} \leftarrow \text{supporters} \cup \{v\}$ 
12:   return  $w(\text{supporters})$ 
13: function MAJORITY_FORK_CHOICE( $\Sigma, \text{anchor}, \text{tree}, r$ )
14:    $\text{total} \leftarrow w(\{v \in V : \text{LATEST}(\Sigma, v, r) \neq \perp\})$ 
15:   define  $\text{eligible}(B)$  as  $2 \text{SG\_SUPPORT}(\Sigma, r, B) > \text{total}$ 
16:   return  $\text{GHOST}(\text{anchor}, \text{tree}, \text{SG\_SUPPORT}(\Sigma, r, \cdot), \text{eligible})$ 
17: function GET_HEAD( $\Sigma, \text{votes}, s$ )
18:    $\text{anchor} \leftarrow \text{MAJORITY\_FORK\_CHOICE}(\Sigma, B_{\text{gen}}, \Sigma.\mathcal{T}, \text{round}(\Sigma.s))$ 
19:   return  $\text{GOLDFISH\_FORK\_CHOICE}(\Sigma, \text{anchor}, \Sigma.\mathcal{T}, \text{votes}, s)$ 

```

3.4 Duty and handler

ON_TICK gains one line: at $t = a_r$ for the current round r , run SG_VOTE. The proposer and voter of Figure 2 call the GET_HEAD of Figure 4; nothing else in their duties changes. Available confirmation is unchanged.

Figure 5: SG duty and store handler

```

1: function SG_VOTE( $\Sigma$ )
2:    $r \leftarrow \text{round}(\Sigma.s)$   $\triangleright$  runs at  $a_r$ 
3:    $\text{vote} \leftarrow (\ell, r, \Sigma.\text{live\_confirmed})$ 
4:   broadcast  $\text{vote}$ ;  $\text{PROCESS\_SG\_VOTE}(\Sigma, \text{vote})$ 
5: function PROCESS_SG_VOTE( $\Sigma, \text{vote}$ )
6:   if  $\text{vote.round} > \text{round}(\Sigma.s)$  or  $\text{vote} \in \text{sg\_votes}[\text{vote.round}]$  then
7:     return
8:   if  $\text{sg\_votes}[\text{vote.round}]$  holds two distinct votes by  $\text{vote.validator}$  then
9:     return  $\triangleright$  two witness the equivocation; nothing reads a third
10:   add  $\text{vote}$  to  $\text{sg\_votes}[\text{vote.round}]$ ;  $\text{timestamp}(\text{vote}) \leftarrow \Sigma.t$ 

```

In the graded protocol of Section 6, a fresh graded root takes precedence for its one eligible round; otherwise this relative-majority root is the SG anchor. The two roots are alternatives: the protocol

selects one anchor and runs Goldfish once from it.

4 Finality-gadget state transition

The finality gadget extends each non-genesis block with a list of combined attestations and a post-state root. It also defines the weighted quorum threshold

$$q = \left\lceil \frac{2W}{3} \right\rceil.$$

A quorum is a set of validators whose total weight is at least q .

4.1 Combined attestations

Height is a finality counter and is separate from slot. Genesis is justified and finalized at height 0, and every chain starts at height 1. A justification or a progress event increments height by one.

We extend the attestation to

$$a = (\text{validator}, \text{round}, \text{head}, \text{height}, \text{target}, \text{finalize_height}, \text{finalize_target}),$$

adding the finality relevant fields `height`, `target`, `finalize_height`, `finalize_target` to the SG vote. The height pair is a target vote (h, T) with $T \neq \perp$, an empty-target vote $(h, \perp)$, or the empty pair $(\perp, \perp)$. The finality pair is (h_f, T_f) with $T_f \neq \perp$, or the empty pair $(\perp, \perp)$.

Two pair occurrences from one validator are slashable under either condition:

- E1:** one finality pair is (h, T) with $T \neq \perp$, and one height pair is (h, T') with $T' \neq T$, including $T' = \perp$;
- E2:** two height pairs are (h, T) and (h, T') with $T, T' \neq \perp$ and $T \neq T'$.

The conflicting occurrences can be in one attestation. An empty height pair has height $\perp$ and triggers neither condition.

An honest validator emits at most one proposal per slot, one Goldfish vote per slot, and one combined attestation per round.

4.2 Chain state and transition

Every block is evaluated from its parent's immutable post-state:

$$\sigma[B] = \text{STATE_TRANSITION}(\sigma[B.\text{parent}], B).$$

The result is a deterministic function of the chain ending at B .

Definition 4 (Chain state). A chain state is

$$\sigma = (L, s, h, T_h, \text{nj}, \text{target_participation}, \text{progress}, \text{finalize}, J, h_j, F, h_F).$$

L is the latest block, $s = L.\text{slot}$, h is the current height, and T_h is the block that carried the transition into height h . The Boolean `nj` is computed when the chain enters h and remains fixed until the height changes.

The arrays `target_participation`, `progress`, and `finalize` contain one Boolean value per validator. Their quorum sets are derived when used:

$$\begin{aligned} Q_{\text{target}}(\sigma) &= \{i : \sigma.\text{target_participation}[i]\}, & Q_{\text{prog}}(\sigma) &= \{i : \sigma.\text{progress}[i]\}, \\ Q_{\text{finality}}(\sigma) &= \{i : \sigma.\text{finalize}[i]\}. \end{aligned}$$

The pair (J, h_j) is the latest justification on the chain, and (F, h_F) is its latest finalization. Initially

$$L = T_h = J = F = B_{\text{gen}}, \quad s = 0, \quad h = 1, \quad h_j = h_F = 0, \quad \text{nj} = \text{false},$$

and every array entry is false.

Fix constants $K \geq 2$ and $D \geq 1$. On entry into a new height h , after the same transition has applied any finalization, set

$$\text{nj} \leftarrow (K \mid h) \wedge (h - h_F > D).$$

Figure 6: Deterministic finality-gadget state transition

```

1: function STATE_TRANSITION( $\sigma, B$ )
2:    $\sigma.s \leftarrow B.\text{slot}$ 
3:   for all  $a \in B.\text{attestations}$  do
4:      $\sigma \leftarrow \text{PROCESS\_ATTESTATION}(\sigma, a)$ 
5:    $\sigma.L \leftarrow B$ 
6:   return PROCESS_HEIGHT_EVENTS( $\sigma$ )
7: function PROCESS_ATTESTATION( $\sigma, a$ )
8:    $i \leftarrow a.\text{validator}$ 
9:   if  $\sigma.h_j > \sigma.h_F$  and  $\sigma.F \preceq \sigma.J$  and  $a$ 's finality pair =  $(\sigma.h_j, \sigma.J)$  then
10:     $\sigma.\text{finalize}[i] \leftarrow \text{true}$ 
11:   if  $a$ 's height pair =  $(\sigma.h, \sigma.T_h)$  then
12:     $\sigma.\text{target\_participation}[i] \leftarrow \text{true}$ 
13:     $\sigma.\text{progress}[i] \leftarrow \text{true}$ 
14:   else if  $a$ 's height pair =  $(\sigma.h, \perp)$  then
15:     $\sigma.\text{progress}[i] \leftarrow \text{true}$ 
16:   return  $\sigma$ 
17: function PROCESS_HEIGHT_EVENTS( $\sigma$ )
18:   if  $\sigma.h_j > \sigma.h_F$  and  $\sigma.F \preceq \sigma.J$  and  $w(Q_{\text{finality}}(\sigma)) \geq q$  then
19:      $(\sigma.F, \sigma.h_F) \leftarrow (\sigma.J, \sigma.h_j)$ 
20:   if  $\neg \sigma.\text{nj}$  and  $w(Q_{\text{target}}(\sigma)) \geq q$  then
21:      $(\sigma.J, \sigma.h_j) \leftarrow (\sigma.T_h, \sigma.h)$ 
22:      $\sigma.\text{finalize} \leftarrow \text{false}^V$ 
23:     return ADVANCE_HEIGHT( $\sigma$ )
24:   if  $w(Q_{\text{prog}}(\sigma)) \geq q$  then
25:     return ADVANCE_HEIGHT( $\sigma$ )
26:   return  $\sigma$ 
27: function ADVANCE_HEIGHT( $\sigma$ )
28:    $\sigma.h \leftarrow \sigma.h + 1$ ;  $\sigma.T_h \leftarrow \sigma.L$ 
29:    $\sigma.\text{nj} \leftarrow (K \mid \sigma.h) \wedge (\sigma.h - \sigma.h_F > D)$ 
30:    $\sigma.\text{target\_participation}, \sigma.\text{progress} \leftarrow \text{false}^V$ 
31:   return  $\sigma$ 

```

5 Finality gadget in the fork-choice

5.1 Stored states and finality updates

The store adds a block state map $\Sigma.\sigma[\cdot]$ and finality state $(\Sigma.F, \Sigma.h_F, \Sigma.J, \Sigma.h_j, \Sigma.h_{\text{max}})$. It retains every processed block; the live tree is derived below the finalized block,

$$\mathcal{T}_F(\Sigma) = \{B \in \Sigma.\mathcal{T} : \Sigma.F \preceq B\}.$$

Figure 7 extends the block handler: it computes and stores the post-state, then folds it into the finality caches with `UPDATE_FINALITY( $\Sigma, \sigma$ )`. The pair $(\Sigma.J, \Sigma.h_j)$ tracks the lex-greatest justification event compatible with the finalized block. $\Sigma.F$ advances only to a viable proper descendant of itself below $\Sigma.J$, so $\Sigma.F \preceq \Sigma.J$ always holds and finalization never reverts. When it advances, $\Sigma.h_{\max}$ is recomputed inside the new live tree; it otherwise only grows. The figure follows the viability definitions it uses. The processed finality evidence is the derived set

$$\mathcal{E}_F(\Sigma) = \{(\Sigma.\sigma[B].F, \Sigma.\sigma[B].h_F) : B \in \Sigma.\mathcal{T}\}.$$

5.2 Fork choice

Definition 5 (Viability). A live block is viable when it has a live descendant whose state height is at most one below the current maximum:

$$\mathcal{V}(\Sigma) = \{B \in \mathcal{T}_F(\Sigma) : \exists W \in \mathcal{T}_F(\Sigma), B \preceq W, \Sigma.\sigma[W].h \geq \Sigma.h_{\max} - 1\}.$$

Fork choice uses two derived views (Figure 7): `FORK_CHOICE_ROOT( $\Sigma$ )`, the block the walk starts from, and `GET_FILTERED_BLOCK_TREE( $\Sigma$ )`, the viable blocks below it, which limit the selectable children. Goldfish starts at the root even if the root is not in the filtered tree.

At this layer `GOLDFISH_ELIGIBLE` gains one clause: a child whose state height is below $\Sigma.h_{\max} - 1$ is eligible without a majority. Figure 7 shows the extended store handler, the extended gate, and the redefined `GET_HEAD`. Available confirmation runs its own walk from $\Sigma.F$ over $\mathcal{T}_F(\Sigma)$; it uses neither the candidate tree, the SG root, nor the height filter.

Figure 7: FG store and fork choice

```

1: function PROCESS_BLOCK( $\Sigma, B$ )
2:   if  $B.\text{slot} > \Sigma.s$  then
3:     return
4:    $\Sigma.\sigma[B] \leftarrow \text{STATE\_TRANSITION}(\Sigma.\sigma[B.\text{parent}], B)$  ▷ new at this layer
5:   add  $B$  to  $\Sigma.\mathcal{T}$ ;  $\Sigma.\text{timestamp}(B) \leftarrow \Sigma.t$ 
6:   for all  $\text{vote} \in B.\text{gf\_votes}$  do
7:     PROCESS_GOLDFISH_VOTE( $\Sigma, \text{vote}$ )
8:   UPDATE_FINALITY( $\Sigma, \Sigma.\sigma[B]$ ) ▷ new at this layer
9: function UPDATE_FINALITY( $\Sigma, \sigma$ )
10:   $\Sigma.h_{\max} \leftarrow \max(\Sigma.h_{\max}, \sigma.h)$ 
11:  if  $\Sigma.F \preceq \sigma.J$  and  $(\sigma.h_j, \sigma.J.\text{root}) > (\Sigma.h_j, \Sigma.J.\text{root})$  then ▷ lex order
12:     $(\Sigma.J, \Sigma.h_j) \leftarrow (\sigma.J, \sigma.h_j)$ 
13:  if  $\Sigma.F \prec \sigma.F$  and  $\sigma.F \preceq \Sigma.J$  and  $\sigma.F \in \mathcal{V}(\Sigma)$  then
14:     $(\Sigma.F, \Sigma.h_F) \leftarrow (\sigma.F, \sigma.h_F)$ 
15:     $\Sigma.h_{\max} \leftarrow \max\{\Sigma.\sigma[B].h : B \in \mathcal{T}_F(\Sigma)\}$  ▷ the live tree just shrank
16: function FORK_CHOICE_ROOT( $\Sigma$ )
17:  if  $\Sigma.h_{\max} = \Sigma.h_j + 1$  then
18:    return  $\Sigma.J$ 
19:  return  $\Sigma.F$ 
20: function GET_FILTERED_BLOCK_TREE( $\Sigma$ )
21:   $\text{root} \leftarrow \text{FORK\_CHOICE\_ROOT}(\Sigma)$ 
22:  return  $\{B \in \mathcal{V}(\Sigma) : \text{root} \preceq B\}$ 
23: function GOLDFISH_ELIGIBLE( $\Sigma, \text{votes}, s, B$ )
24:   $\text{voters\_count} \leftarrow |\{v \in \mathcal{K}_s : \text{votes holds a vote by } v\}|$ 
25:  return  $\Sigma.\sigma[B].h < \Sigma.h_{\max} - 1$  or  $2 \text{ GOLDFISH\_SCORE}(\text{votes}, s, B) > \text{voters\_count}$  or  $B.\text{slot} = \Sigma.s$ 
26: function GET_HEAD( $\Sigma, \text{votes}, k$ )
27:   $\text{root} \leftarrow \text{FORK\_CHOICE\_ROOT}(\Sigma)$ ;  $\text{tree} \leftarrow \text{GET\_FILTERED\_BLOCK\_TREE}(\Sigma)$ 
28:   $\text{anchor} \leftarrow \text{MAJORITY\_FORK\_CHOICE}(\Sigma, \text{root}, \text{tree}, \text{round}(\Sigma.s))$ 
29:  return GOLDFISH_FORK_CHOICE( $\Sigma, \text{anchor}, \text{tree}, \text{votes}, k$ )

```